RASHTREEYA SIKSHANA SAMITHI TRUST

RV COLLEGE OF ENGINEERING®

(Autonomous Institution Affiliated To VTU, BELAGAVI)
R.V. Vidyaniketan Post, Mysuru Road,
BENGALURU - 560 059

MASTER OF COMPUTER APPLICATIONS

RV College of Engineering®

Lab Manual for First Semester

Object Oriented Programming Using Java

MCA113IA

Faculty In-charge

Dr. JASMINE K S
&
Prof. SARAVANAN C

USN	
NAME	
Academic Year	**2024-2025**

Acknowledgment

Sincere thanks to

Mr.Prashanth K
Dr. B. H. Chandrashekar
Dr. Vishal C
&
Dr. Mohan Aradhya

For their contribution in compiling the lab manual

RASHTREEYA SIKSHANA SAMITHI TRUST (RSST)

The last decade of pre-independence India was marked with several initiatives and entrepreneurial ventures. One such unique venture was the founding of Rashtreeya Sikshana Samithi Trust (RSST), by **Sri. M.C. Sivananda Sarma**, a freedom fighter and a scholar in the year 1940. The organization started with a noble mission to impart quality education to all sections of the society, without any favour or bias towards any one. Today, the charitable trust provides avenue for quality education, catering to a wide sector of educational needs, starting from kindergarten to post graduate education as well as research in advanced engineering, medical and architecture domains. The sustained growth and success is due to the dedicated efforts of the management and their continued commitment to the founder‟s vision, mission, quality, continuous improvement and concern towards social responsibility. Today the trust is managed by a very distinguished board of trustees led by **Dr. M.P. Shyam, President & Chairman Governing body, Shri Panditharadhya, Vice Presidents, Shri K.G. Subbarama Setty and Shri D.P. Nagaraj, Hon. Joint Secretary**. The board of trustees recognizes the importance of holistic education and a need for a learning environment that nurtures healthy competition and innovation. Rashtreeya Sikshana Samithi Trust manages over twenty educational institutions including schools, colleges offering degree, post graduate programs and doctoral programs in different specialties. RSST continuously strives to create state of art infrastructure, recruit excellent faculty and facilitate efficient administration in all its institutions to provide congenial ambience for learning. Today, RSST is regarded as one of the finest and the best managements for education in the country.

RV COLLEGE OF ENGINEERING®

Marching towards Excellence in Education, Research and Innovation

Rashtreeya Vidyalaya College of Engineering (RVCE), established in 1963 is one of the earliest self-financing Engineering colleges in the country. RV College of Engineering is the flagship institution of RSST. The institution provides opportunities to all sections, including the under privileged, differently abled and socially marginalized people to gain engineering skills through various programs like WEST-Women Empowerment and Skill Training etc. RVCE is rated amongst the top five self-financing Engineering colleges in the country. Some magazines have rated it as the best among private institutions in the country, including in terms of best Return on Investment for a student. RVCE is a preferred destination for top ranking aspirants, both for UG, PG and Doctoral programs. RVCE is an Autonomous college, affiliated to Visvesvaraya Technological University (VTU) Belagavi. The institution has its own Academic Council which is empowered to approve the academic curriculum as

suggested by the Board of Studies of various programs and as per the VTU guidelines for autonomous institutions. RVCE currently offers 14 Bachelor, 16 Master programs and 15 centers of research to carry out research and consultancy activities in the departments. All UG programs have been accredited multiple times. Some of the P.G. programs have been accredited and other eligible PG programs have applied for accreditation.

The Institution currently has a student strength of about 5600, faculty strength of more than 400, technical and administrative staff of 241 and about 230 research scholars are pursuing PhD. The institution has set itself a Vision "Leadership in Technical Education, Interdisciplinary Research & Innovation, with a focus on Sustainable and Inclusive Technologies". All the departments are aligned to the vision of sustainable and inclusive technology development, with a focus to contribute to both technological leadership of the nation and welfare of all sections of the society. RVCE is rapidly expanding its R&D activity and Industry academic collaborations.

VISION

Leadership in Technical Education, Interdisciplinary Research, Innovation, with a Focus on Sustainable and Inclusive Technologies.

MISSION

- To deliver Outcome Based Quality Education, emphasizing on experiential learning with state-of-the-art infrastructure
- To create a conducive environment for interdisciplinary research and innovation
- To develop professionals through holistic education focusing on individual growth, discipline, integrity, ethics and social sensitivity
- To nurture industry-institution collaboration leading to competency enhancement and entrepreneurship.
- To focus on technologies that are sustainable and inclusive, benefiting all sections of the society

PROFILE OF THE DEPARTMENT

The Department of Master of Computer Applications was established in the year 1997 and is the first PG programme started in RVCE. Currently, the department offers one PG programme MCA and Research programmes viz M.Sc by Research and Ph.D. Degree in various specializations of Computer Applications, Affiliated to Visvesvaraya Technological University, Belagavi. The MCA programme received academic autonomy in the year 2016. The MCA programme is accredited four times

by National Board of Accreditation, New Delhi. The department is accredited by NBA for five years - 2021-2022 to 2025-2026. The Department is equipped with state-of-the-art infrastructure and computing equipment supported by high-speed Ethernet and wireless networks. Students are encouraged to take up interdisciplinary research and internship in various domains and the overall excellence of development of the students is reflected in the exemplary placement record of the department. The faculty is diverse, committed, highly qualified with Doctorates in various specializations. Quality education, blended with rich research experience along with hands-on teaching is provided to students. The faculties are engaged in active research and consultancy works and the department has generated 2.72 crores under projects, consultancy and training. The current Intake for MCA Programme is 120. The duration of the course, from 2020 admission, is two years (Four Semesters) and curriculum is aimed at interspersing wide spectrum of practice and knowledge in the computing domain in the latest and emerging technologies with inputs and active support from industry experts and strong Alumni.

VISION

Pioneering in ICT Enabled Quality Education and Research with a focus on Sustainable and Inclusive Applications.

MISSION

M1 To adapt novel methodologies for quality education through experiential learning

M2 To empower students with continuous, holistic education, emphasizing on discipline, ethics and social commitment

M3 To become a vibrant knowledge center for research and softwaredevelopment

M4 To continuously build capacity steering towards industry- institute collaborative research and entrepreneurial competencies

M5 To utilize and develop free and open source software tools for sustainable and inclusive growth

Program Educational Objectives (PEO)

MCA graduates will be able to:

PEO1**:** Practice software engineering principles and standards to develop software to meet customer requirements across verticals

PEO2: Contribute to build sustainable and inclusive applications using mathematical, simulation and meta-heuristic models

PEO3: Demonstrate entrepreneurial qualities through individual competence and team work

PEO4: Achieve successful professional career with integrity and societal commitments leading to lifelong learning

Program Outcomes (PO)

MCA graduates will be able to:

PO1 (Foundation Knowledge): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyse problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (Project Management and Finance): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware

PO8 (Life-long learning): Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

Program Specific Criteria (PSC)

The MCA program will enable the students, by the time they graduate to:

PSC1: Explain the principles of mathematics, computing and business foundations

PSC2: Demonstrate the use of software tools and technologies relevant to various verticals

PSC3: Design and develop software products, processes and systems for real world situations

Program Specific Outcomes (PSO)

MCA graduates will be able to:

PSO1: Solve real world computing system problems of various industries by understanding and applying the principles of mathematics, computing techniques and business concepts

PSO2: Design, test, develop and maintain desktop, web, mobile and cross platform software applications using modern tools and technologies.

RV COLLEGE OF ENGINEERING®, BENGALURU – 560059

(Autonomous Institution Affiliated to VTU, Belagavi)

Department of Master of Computer Applications

CERTIFICATE

This is to certify that Mr./Ms ……

USN ……..….... of **1st Semester** Master of Computer Applications program has satisfactorily completed the course of experiments in practical **Object Oriented Programming using Java** (**MCA113IA)** prescribed for the academic year **2024 – 2025**.

LAB MARKS	
Max	**Obtained**
50	

Signature of Student **Signature of faculty In-Charge** **Signature of Director**

EVALUATION SHEET

Week	Prog. No.	Program Title	Date of Execution	Marks	Signature of Lab In-charge
1.	**1.**	Write a Java program to demonstrate the concepts a. Scope of variable b. this keyword			
2.	**2.**	Write a Java program to demonstrate the concepts a. Constructor overloading and method overloading b. Static keyword			
3.	**3.**	Complete the following: a) Create a package named shape. b) Create classes in the package representing common shapes like Square, Triangle, and Circle. Import and compile these classes in other program to find the area of different shapes.			
4.	**4.**	Write a Java program to demonstrate the concepts a. Abstraction b. Run Time Polymorphism			
5.	**5.**	.Write a Java program to demonstrate multiple interfaces			
6.	**6.**	Write a Java program to demonstrate user defined Exception Handling using throw, throws			
7.	**7.**	Write a Java program that demonstrates the Thread Life Cycle			
8.	**8.**	Write a Java code to demonstrate producer & consumer problem using thread wait & notify methods.			
9.	**9.**	Write a Java program to demonstrate Generic classes and methods			
10.	**10.**	Demonstrate RESTful web API to create a web resource which can be accessed using REST URI‟s and demonstrate the concept of GET, POST, PUT & DELETE			
Lab Manual Marks				**/100**	
Lab Manual Marks (Reduced to 30)				**/30**	
Lab Internals				**/50**	
Lab Internals (Reduced to 10)				**/10**	
Open-ended Lab Experiment				**/10**	
Total				**/50**	

Student Signature

INDEX

Program No	Program Title	Page No
I	List of lab programs for the course	12
II	Guidelines, Do's and Don'ts	15
III	Introduction to Java Programming	17
1.	Write a Java program to demonstrate the concepts a. Scope of variable b. this keyword	20
2.	Write a Java program to demonstrate the concepts a. Constructor overloading and method overloading b. Static keyword	23
3.	Complete the following: a) Create a package named shape. b) Create classes in the package representing common shapes like Square, Triangle, and Circle. Import and compile these classes in other program to find the area of different shapes.	26
4.	Write a Java program to demonstrate the concepts a) Abstraction b) Run Time Polymorphism	28
5.	Write a Java program to demonstrate multiple interfaces	31
6.	Write a Java program to demonstrate user defined Exception Handling using throw, throws	34
7.	Write a Java program that demonstrates the Thread Life Cycle	38
8.	Write a Java code to demonstrate producer & consumer problem using thread wait & notify methods.	40
9.	Write a Java program to demonstrate Generic classes and methods	43
10.	Demonstrate RESTful web API to create a web resource which can be accessed using REST URI"s anddemonstrate the concept of GET, POST, PUT & DELETE	46
11.	**Open-ended Lab Experiment:** Students are required to identify and solve allpossible solutions for any three programs listed	48
Appendix A	**VIVA QUESTIONS**	51

Laboratory Component

CIE Marks(Lab):	50	SEE Marks(Lab):	50
Hr/Week:	2	SEE Hours:	03

Prog. No.	Program Title
1.	Write a Java program to demonstrate the concepts a. Scope of variable b. this keyword
2.	Write a Java program to demonstrate the concepts a. Constructor overloading and method overloading b. Static keyword
3.	Complete the following: a) Create a package named shape. b) Create classes in the package representing commonshapes like Square, Triangle, and Circle. Import and compile these classes in other program tofind the area of different shapes.
4.	Write a Java program to demonstrate the concepts a. Abstraction b. Run Time Polymorphism
5.	Write a Java program to demonstrate multiple interfaces
6.	Write a Java program to demonstrate user defined Exception Handling using throw, throws
7.	Write a Java program that demonstrates the Thread Life Cycle
8.	Write a Java code to demonstrate producer & consumer problem using thread wait & notify methods.
9.	Write a Java program to demonstrate Generic classes and methods
10.	Demonstrate RESTful web API to create a web resource which can be accessed using REST URI‟s anddemonstrate the concept of GET, POST, PUT & DELETE

Scheme of Continuous Internal Evaluation (CIE) for Practical:

CIE for the practical will be based on the performance of the student in the laboratory every week for 10 marks for every experiment. Finally, the weekly evaluated marks will be consolidated for 30 marks. One test will be conducted at the end of the semester for 50 marks and will be reduced to 10 marks and Open-ended Lab Experiment will be evaluated for 10 marks. The total marks for CIE (Practical) will be for 50 marks. No change of program is allowed.

Scheme of Semester End Evaluation (SEE) for Practical:

SEE for the practical will be based on writing proper program, execution and proper results and viva for 50 marks. No change of program is allowed.

Break-up Scheme for Continuous Internal Evaluation for Laboratory

Continuous Internal (Evaluation Laboratory)	Total Marks	Scale Down to
Performance of the student weekly in laboratory for every program (10 Marks)	For 10 Lab Programs 10 x 10 = 100 Marks	30
Internal Test(50 Marks) Write Up(20% of 50 Marks) Program Execution (60% of 50 Marks) Viva-Voce(20% of 50 Marks)	Total : 50 Marks	10
Lab Record marks(30 Marks) & Internal Test (10 Marks)	Total: 40 Marks	40
Open-ended Lab Experiment	Total: 10 Marks	10
	TOTAL CIE	50

Rubrics for Each Practice Sessions

SI NO	Rubrics	Max.Marks
1	Understanding of Concepts and Logic	**2**
2	Approach Towards the Problem	**2**
3	Conduction and Execution: Input & Output for All Possible Cases	**3**
4	Program Analysis & Applications	**2**
5	Communications & Confidence Level	**1**
Total		**10**

Rubrics for Open-ended Lab Experiment

SI NO	Rubrics	Max.Marks
1	Understand the need and fundamental concepts of object orientation in real world applications	**2**
2	Identify and apply the object orientation in Java concepts for given problem	**3**
3	Demonstrate all possible solutions and validations for given problem scenario	**3**
4	Justify the oops concepts used in solving the problem	**2**
Total		**10**

Rubrics for Scheme of Evaluation of Lab Experiments

CONTINUOUS INTERNAL EVALUATION					
RUBRICS FOR LAB PROGRAM CONDUCTION AND EXECUTION					
Sl.No	**Criteria**	**Marks Allotted (Max: 7 marks)**			**Total Marks**
		Excellent	**Good**	**Poor**	
1	Knowledge / Understanding of Concepts and Logic (2 marks)	Demonstrates the ability of describing concepts & logic (1.5- 2.0)	Demonstrates considerable ability in describing the concepts & logic (1.4 - 1.0)	Unable to demonstrate, describe and illustrate the concepts & logic (0)	7
2	Algorithmic Approach (2 marks)	Excellent/different Analysis, design and development to get expected output (1.5- 2.0)	Good Analysis design and development to get expected output (1.4 - 1.0)	Poor Analysis design and development toget expected output (0)	
3	Conduction and Execution of the program and Output (3 marks)	Conduct the program by inputting correct data and excellent presentation of the results (2-3)	Conduct the program by inputting correct data and fair presentation (1-1.9)	Conduct the program by inputting correct data and poor presentation of the results (<= 1)	
VIVA-VOCE					
CIE RUBRICS MARKS ALLOTTED (MAX : 3 MARKS)					
4	Program Analysis (1 mark)	Answers questions and demonstrates complete understanding of the working (1.0)	Answers Questions & demonstrates good understanding of the working (0.5)	Answers to questions and demonstrates poor understanding of the working (0)	3
5	Application (1 mark)	Sound knowledge about applicability (1.0)	Demonstrates good knowledge about applicability (0.5)	Demonstrates very limited knowledge about applicability (0)	
6	Communication (1 mark)	Communicates all ideas clearly about method/ procedure(1.0)	Explains the procedure without clear understanding (0.5)	No idea about the procedure / method (0)	

RV COLLEGE OF ENGINEERING®, BENGALURU – 560059
(Autonomous Institution Affiliated to VTU, Belagavi)

Department of Master of Computer Applications

Object Oriented Programming using Java – MCA113IA

General Guidelines

- Use the **Eclipse IDE**
- Programs must be indented appropriately to enhance readability and easy maintenance
- Students are required to explore and apply Java based programming solutions
- Students are encouraged to experiment with alternative approaches to implement program requirements

Laboratory Guidelines

Do's

- Arrive at the laboratory well-prepared
- Submit your lab records to the staff in charge and sign the logbook upon entering the laboratory
- Strictly adhere to the lab exercise cycles as outlined by the department. Non-compliance may result in mark deductions
- Use the login credentials (if any) assigned to you consistently
- Complete any pending exercises only after finishing the assigned regular exercises

Don'ts

- Avoid unnecessary movement within the laboratory during the lab session
- Do not tamper with system files or attempt to access the server without authorization
- Writing data sheets or lab records in the laboratory is strictly prohibited
- Do not switch systems without prior approval from the lab staff
- Refrain from assisting or teaching other students during the lab session

Prerequisites

Students should have the basic knowledge of Programming Concepts.

Course Learning Objectives

Graduates shall be able to

1. Understand and apply object-oriented principles to design and implement solutions for real-world Java applications.
2. Develop modular and reusable code structures using Java programming techniques.
3. Analyze real-world problems and design robust solutions using advanced Java concepts.
4. Utilize Java's Collections Framework and Generics to create efficient, scalable, and type-safe data structures for software development.

Expected Course Outcomes

After going through this course, the student will be able to:

CO1 Apply object-oriented principles to design and implement real-world Java applications.

CO2 Design Java-based solutions to achieve modular and reusable code structures.

CO3 Analyze real-world problems and implement robust solutions using Java concepts.

CO4 Demonstrate Java's Collections Framework and Generics to create efficient, scalable, and type-safe data structures for software applications.

RV COLLEGE OF ENGINEERING®, BENGALURU – 560059

(Autonomous Institution, Affiliated to VTU, Belgavi)

Object Oriented Programming Using Java -MCA113IA

INTRODUCTION

Object Oriented Programming Using Java covers the basic topics of Java fundamentals and Restful API.

The JAVA Language

The Origin of Java: Java was conceived by James Gosling and others at Sun Microsystems in 1991. This Language was initially called "Oak" but was renamed as "Java" in 1995.

Java's Contribution to the internet:

Java Applets: And applet is a special kind of java program that is designed to be transmitted over the internet and automatically executed by a Java-compatible web browser.

Security: Java achieved this protection by confining an applet to the Java execution environment and not allowing it access to other parts of the computer.

Portability: Portability is a major aspect of the Internet because there are many different types of computers and operating systems connected to it.

Java's Solution the Bytecode: Bytecode is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM).

Evolution of Java:

The Key attributes of OOP

Encapsulation: Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.

Polymorphism: Polymorphism (from Greek, meaning "many forms") is a feature that allows one interface to be used for a general class of actions.

Inheritance: Inheritance is the process by which one object acquires the properties of another object.

The Java Development Kit:

The JDK supplies two primary programs. The first is javac, which is the java compiler. It compiles source code into bytecode. The second is java, sometimes referred to as the application launcher. This is the program used for running a java program. It operates on the bytecode, using the JVM to execute programs.

A First Simple Program

```
/*
This is a simple Java program.
Call this file "Example.java".
```

```
*/
class Example {
// Your program begins with a call to main().
public static void main(String args[]) {
System.out.println("This is a simple Java program.");
}
}
```

First Sample Program Line by Line:

```
/*
This is a simple Java program.
Call this file "Example.java".
*/
```

This is a multiline comment.

The next line in the program is the single-line comment, shown here:

// Your program begins with a call to main().

The next line of code is shown here:

public static void main(String args[]) {

All Java applications begin execution by calling main( ). The full meaning of each part of this line cannot be given now, since it involves a detailed understanding of Java‟s approach to encapsulation. However, since most of the examples in the first part of this book will use this line of code, let‟s take a brief look at each part now.

Design patterns

Design patterns are reusable solutions to common software design problems. They provide a way to solve recurring problems in software development and promote best practices for designing and structuring code. In Java, there are several widely used design patterns. Here are some of the most common design patterns in Java:

Singleton Pattern: Ensures that a class has only one instance and provides a global point of access to it.

Factory Pattern: Provides an interface for creating objects without specifying their concrete classes. It allows subclasses to decide which class to instantiate.

Builder Pattern: Allows the construction of complex objects step by step. It separates the construction of an object from its representation, allowing the same construction process to create different representations.

Prototype Pattern: Creates new objects by cloning existing ones, thus avoiding the need for explicit construction.

Adapter Pattern: Converts the interface of a class into another interface that clients expect. It allows incompatible classes to work together by converting the interface of one class into an interface compatible with the client code.

Observer Pattern: Defines a one-to-many dependency between objects. When one object changes its state, all its dependents are notified and updated automatically.

Decorator Pattern: Adds new functionality to an existing object dynamically without altering its structure. Proxy Pattern: Provides a surrogate or placeholder for another object to control access to it. It allows adding an extra layer of indirection to support additional functionality.
Strategy Pattern: Defines a family of interchangeable algorithms and encapsulates each one. It allows the algorithms to be selected at runtime, providing a way to switch between different strategies dynamically.
Template Method Pattern: Defines the skeleton of an algorithm in a base class and lets subclasses override certain steps of the algorithm without changing its structure.

Threads in Java

In Java, a thread represents a sequential flow of control within a program. It allows concurrent execution of multiple tasks or parts of a program, enabling parallelism and improving performance. Threads can run independently and concurrently, allowing different parts of a program to execute simultaneously.

To work with threads in Java, there are two main approaches:

Extending the Thread class: can create a new class that extends the Thread class and override the run() method. The run() method contains the code that will be executed when the thread starts running. To start the thread, create an instance of subclass and call its start() method.

Implementing the Runnable interface: Alternatively, can implement the Runnable interface, which defines a single method called run(). This approach is often preferred because it allows better separation of concerns. Create a class that implements Runnable and provide the implementation for the run() method. Then, pass an instance of class to a Thread object and call its start () method to start the thread.

Program No. 1

Write a Java program to demonstrate the following concepts

a. Scope of a variable b. this keyword

a. **Scope of Variables**

In Java, the **scope of a variable** defines the part of the program where the variable can be accessed, used, and modified. It determines the variable's lifetime and visibility within the code. Java variables can have different scopes depending on where they are declared.

- Instance Variables: Declared inside the class but outside any method, constructor, or block.
- Local Variables: Declared within a method or a block and accessible only within that method/block.
- Block Variables: Declared inside blocks like if, for, or while.

b. **this Keyword**

- Used to refer to the current object.
- Helps resolve ambiguity when a method parameter or local variable has the same name as an instance variable.

Question: write a java program to implement a calculator program that performs arithmetic operations. This program should demonstrate the difference between local variables, instance variables, static variables and block variables.

Step wise Execution Procedure:

Students should demonstrate the **Scope of a variable** and **this** keyword, by solving the above given question.

Step 1: Create a class Calculator & define parameterized constructor of Calculator class

Step 2: **Define a method calculateSum**

Step 3: Define static variable result & instance variables in Calculator Class, local variable block variable within calculateSum method

Step 4: **Create an instance of Calculator class**

Step 5: **Demonstrate Scope of variables and this keyword**

Expected Output:

- The program should demonstrate the correct usage of Local variable, Block variable and Instance variable

Sl.No	Rubrics for Practice Sessions		Max. Marks	Marks Obtained
1	**Conduction & Execution**	Understanding of Concepts and Logic	**2**	
2		Approach Towards the Problem	**2**	
3		Conduction and Execution: Input & Output for All Possible Cases	**3**	
4	**Viva Voce**	Program Analysis & Applications	**2**	
5		Communication & Confidence Level	**1**	
		TOTAL	**10**	
		Signature		

Program No. 1

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

Program No. 2

Write a Java program to demonstrate the following concepts

a. Constructor overloading and method overloading **b**. static keyword

A **constructor** in Java is a special method used to initialize objects of a class. It is automatically called when an object of the class is created. Constructors set up the initial state of an object by assigning values to its fields or performing any required setup tasks.

Types of Constructors in Java

a. Default Constructor: Automatically provided by Java if no constructor is explicitly defined.

b. Parameterized Constructor: Allows passing arguments to set the initial state of an object.

c. No-Argument Constructor: A constructor without any parameters, explicitly defined by the programmer. Often used to initialize default values.

d. Copy Constructor: Used to create a new object as a copy of an existing object.

a. Constructor Overloading

Java allows multiple constructors in the same class with different parameter lists (number, type, or order). This is called **constructor overloading**.

Method Overloading in Java is a feature that allows a class to have multiple methods with the **same name** but with **different parameter lists**. It enables methods to perform similar tasks but on different types or numbers of inputs, increasing code readability and reusability.

d. Static keyword

The static keyword in Java is used to denote that a particular member (field, method, or block) belongs to the class rather than to any specific instance of the class. This means you can access a static member without creating an object of the class.

Question: write a java program to develop a banking application. Use constructors and methods to handle different scenarios for account creation and performing transactions.

Students should demonstrate both **constructor overloading** and **method overloading** for the given problem scenario.

Step wise Execution Procedure:

1. Create a class Bankaccount
2. Define multiple constructors for different use cases to demonstrate '**constructor**' Overloading as given below
   - Only Bank account name (default account type is "Savings" with balance 0).
   - Name and an initial deposit.
   - Name, initial deposit, and account type.
3. Perform transactions using the `deposit` method
4. Apply the method overloading concept to '**deposit**' method
   - Deposit by specifying the amount only.
   - Deposit by specifying both amount and transaction type (e.g., "Cash" or "Cheque").
5. Use of static Keyword
   - Use a static variable to keep track of the total number of accounts created.
   - Use a static method to display the total accounts.

Expected Output:

The program should demonstrate **Constructor Overloading** for object creation for different scenarios and **Method Overloading** to handle different types of deposits.

Sl.No	**Rubrics for Practice Sessions**		**Max. Marks**	**Marks Obtained**
1	**Conduction & Execution**	Understanding of Concepts and Logic	**2**	
2		Approach Towards the Problem	**2**	
3		Conduction and Execution: Input & Output for All Possible Cases	**3**	
4	**Viva Voce**	Program Analysis & Applications	**2**	
5		Communication & Confidence Level	**1**	
		TOTAL	**10**	
		Signature		

Program No. 2

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated bythe staff in-charge

Program No. 3

Write a Java program to demonstrate the Package concepts for the given scenario.

Complete the following:

1. Create a package named shape.
2. Create classes in the package representing common shapes like Square,Triangle, and Circle.
3. Import and compile these classes in other program to find the area ofdifferent shapes

Step wise Execution Procedure:

Step 1: Creating the package with class.

1.1: In the project tab on source packages→ right click → new → Java Class.

A wizard for the class will be popped up and in that give the class name as square. In the package field write Shapes in the same window. Repeat the step 1.1 three times for creating Triangle and Circle.

Step 2: Importing the package into the main class:

2.1: import the package Shapes

2.2: Create the objects of the classes

2.3: Call the respective methods of the classes.

Step 3: Right click on the source of the main class → click on Run File

Expected Output:

The program should display the methods called from the respective classes

Sl.No	Rubrics for Practice Sessions		Max. Marks	Marks Obtained
1	**Conduction & Execution**	Understanding of Concepts and Logic	**2**	
2		Approach Towards the Problem	**2**	
3		Conduction and Execution: Input & Output for All Possible Cases	**3**	
4	**Viva Voce**	Program Analysis & Applications	**2**	
5		Communication & Confidence Level	**1**	
		TOTAL	**10**	
		Signature		

Program No. 3

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

Program No. 4

Write a Java program to demonstrate the concepts
i) Abstraction ii) Runtime Polymorphism

Introduction:

Abstraction is a concept that allows representing the essential features of an object while hiding the implementation details. It helps in creating reusable code and simplifies complex systems by breaking them down into smaller, more manageable parts.

Abstract Keyword is being used to create a abstract class:
Syntax: abstract class_name{
    //code goes here
}

Runtime polymorphism, also known as dynamic method dispatch, is a feature that allows a subclass to provide different implementations of a method defined in its superclass. The appropriate method implementation is determined at runtime based on the actual type of the object.

1) **Abstraction:** Example program for creating an abstract class and exploring the different functionality

Example program:

Question: write a java program to create an abstract class printer and implement all the methods using the concrete classes.

Step 1: Create a abstract class called the printer with the methods like configuration and display

Step 2: Create two concrete classes "Dot-matrix" and "LaserJet". Implement the abstract class printer and its methods with respect to the abstract class.

Step 3: A Main class with a menu driven program for implementing the abstraction

Expected Output:
The program should display the details of the Dot-matrix and LaserJet class details with an implementation of the abstract class.

2) **Runtime polymorphism:**

Example program that showcases runtime polymorphism using a class hierarchy of vehicles

Question: write a java program:

a. **Create a vehicle interface with a concrete class as car and motorcycle to implement the vehicle interface methods**

b. **Demonstrate runtime polymorphism with an abstract class**

Step wise Execution Procedure:

Step 1: Creating abstract class vehicle.

Step 2: Create method Drive ( )

Step 3: Create two concrete classes, Car and Motorcycle, which inherit from the Vehicle class. Each subclass provides its own implementation of the drive () method

Step 4: Create Main class, create objects of type Vehicle but instantiate them as Car and Motorcycle

Step 5: Demonstrate the concept of polymorphism

Expected Output:

The program should display the common interface called from the respective classes.

Sl.No	**Rubrics for Practice Sessions**		**Max. Marks**	**Marks Obtained**
1	**Conductionn & Execution**	Understanding of Concepts and Logic	**2**	
2		Approach Towards the Problem	**2**	
3		Conduction and Execution: Input & Output for All Possible Cases	**3**	
4	**Viva Voce**	Program Analysis & Applications	**2**	
5		Communication & Confidence Level	**1**	
		TOTAL	**10**	
		Signature		

Program No. 4

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

Program No. 5

Write a Java programs to demonstrate the concepts of multiple interfaces

In Java, an **interface** is a reference type similar to a class that is used to define a set of abstract methods that a class must implement. It provides a way to achieve **abstraction** and **multiple inheritance**.

Need of Multiple interfaces

- Multiple interfaces in Java are essential for addressing specific design needs and enhancing flexibility, modularity, and scalability in software development.
- Java does not support multiple inheritance through classes (to avoid the "diamond problem").
- By using multiple interfaces, a class can inherit behavior from multiple sources without conflicts.

Example program that showcases the concept of multiple interfaces:
Question:

In a workplace, Employee's details are categorized into two distinct domains: **personal information** (e.g., name, age) and **official information** (e.g., designation, department). Efficient management of these details requires a structured and modular approach to represent and process the data. Write a java program to solve the process using multiple interfaces.

Step wise Execution Procedure:
Students should demonstrate use of **Multiple Interfaces**, by solving theabove given question.

Step 1:
- Create an interface Personal to display personal details
- Create another interface Official to display official details

Step 2: Create an Employee class which implements both theInterfaces

Step 3: Define the interface methods in Employee class

Step4: Create a main class with objects of Employee class

Step 5: Demonstrate multiple interfaces

Expected Output:

- The program should display the both Personal and Official Details of Employee with proper input

Sl.No	Rubrics for Practice Sessions		Max. Marks	Marks Obtained
1	**Conduction & Execution**	Understanding of Concepts and Logic	**2**	
2		Approach Towards the Problem	**2**	
3		Conduction and Execution: Input & Output for All Possible Cases	**3**	
4	**Viva Voce**	Program Analysis & Applications	**2**	
5		Communication & Confidence Level	**1**	
		TOTAL	**10**	
		Signature		

Program No. 5

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

Program No. 6

Write a Java program that demonstrates User defined Exception Handling using throw, throws

Exceptions in Java

An **exception** is an event that disrupts the normal flow of a program during its execution. Exceptions are objects that describe an error or unexpected event.

Exceptions in Java are part of the java.lang.Exception class, which is a subclass of Throwable.

Types of exceptions:

1. **Checked Exceptions:** Exceptions checked at compile time (e.g., Exception, SQLException).
2. **Unchecked Exceptions (Runtime Exceptions):** Exceptions checked at runtime (e.g., NullPointerException, ArrayIndexOutOfBoundsException).
3. **Errors:** Serious problems that applications should not handle (e.g., OutOfMemoryError, StackOverflowError).

Java provides specific keywords to handle exceptions:

- try: Block to define code that may throw an exception.
- catch: Block to handle exceptions.
- finally: Block to execute code irrespective of exceptions.
- throw: Used to explicitly throw an exception.
- throws: Used to declare exceptions that a method may throw.

User-Defined Exceptions

Java allows developers to create custom exceptions by extending the Exception or RuntimeException class.

throw:

- The throw keyword is used to explicitly throw an exception, either predefined or user-defined.
- It is typically followed by an instance of an exception class.

Syntax: throw new ExceptionType("Error Message");
Eg: throw new ArithmeticException("Division by zero not allowed.");

throws:

- The throws keyword is used in a method declaration to specify which exceptions the method may throw.
- It helps propagate exceptions to the calling method for handling.

Syntax: returnType methodName(parameters) throws ExceptionType {
// Method body
}

Eg:
public void readFile(String filePath) throws IOException {
// Code that may throw IOException
}

Question: Write a Java program to demonstrate User defined Exception Handling for the scenario given.

A user wants to input their age for a service requiring a minimum age of 18. If the input age is below the threshold, the program should raise a **custom exception** (e.g., InvalidAgeException) with a meaningful error message.

i. Check if a user-provided value (e.g., age or divisor) meets specific criteria (e.g., age ≥ 18, divisor ≠ 0).
ii. Use both predefined exceptions and custom (user-defined) exceptions to signal errors.
iii. Use the throws keyword in method declarations to pass exceptions to the calling method for handling.
iv. Use the throw keyword to raise exceptions when invalid conditions are detected.

Setup wise Execution Procedure:

Step 1: Define a Custom Exception

- Create a class named `InvalidAgeException` that extends the `Exception` class.
- Provide a constructor in this class to accept a custom error message.

Step 2: Write a Method to Validate Age

- Define a method validateAge(int age) to check if the age meets the required condition (age >= 18).
- Use the throws keyword in the method declaration to indicate it may throw an InvalidAgeException.
- Use the throw keyword inside the method to explicitly raise an InvalidAgeException if the condition fails.

Step 3: Write a Method to Perform Division

- Define a method divideNumbers(int dividend, int divisor) to check if the divisor is not zero.
- Use the throw keyword to raise a predefined ArithmeticException if the divisor is zero.

Step 4: Main Method to Handle Input and Exceptions

- Use a try-catch block to call the validateAge and divideNumbers methods.
- Catch and handle both predefined exceptions (ArithmeticException) and the custom exception (InvalidAgeException) with meaningful error messages.

Expected output:

- Age below 18 to trigger the InvalidAgeException.
- Divisor as 0 to trigger the ArithmeticException.
- Valid inputs to ensure normal execution.

Sl.No	Rubrics for Practice Sessions		Max. Marks	Marks Obtained
1	**Conduction & Execution**	Understanding of Concepts and Logic	**2**	
2		Approach Towards the Problem	**2**	
3		Conduction and Execution: Input & Output for All Possible Cases	**3**	
4	**Viva Voce**	Program Analysis & Applications	**2**	
5		Communication & Confidence Level	**1**	
		TOTAL	**10**	
		Signature		

Program No. 6

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

Program No. 7

Write a Java program that demonstrated the Thread Life Cycle.

In Java, threads go through different states during their lifecycle. The Thread class provides methods and mechanisms to manage the lifecycle of threads. Here are the different states of a thread's lifecycle in Java

Methods to be familiarized with are yield(), sleep(), start(), stop().

Question: Write a Java program to demonstrate life cycle of threads.

Setup wise Execution Procedure:

Step1: Create a two classes named as the Thread1 and Thread2

Step2: Thread1 generates the numbers from 1 to 5 with a sleep time of 2 seconds

Step3: Thread2 generates the sequences of numbers from 10 to 15 with a sleep time of 2 seconds

Step 4: Create a main program to start both the threads and demonstrate the working of the block and recall of each thread from time to time.

Expected output:

Demonstration should show the threads start and intercommunication like block and run the other thread.

Sl.No	Rubrics for Practice Sessions		Max. Marks	Marks Obtained
1	**Conduction & Execution**	Understanding of Concepts and Logic	**2**	
2		Approach Towards the Problem	**2**	
3		Conduction and Execution: Input & Output for All Possible Cases	**3**	
4	**Viva Voce**	Program Analysis & Applications	**2**	
5		Communication & Confidence Level	**1**	
		TOTAL	**10**	
		Signature		

Program No. 7

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

Program No. 8

Write a Java code to demonstrate producer & consumer problems using threadwait & notify methods

Threads communicate primarily by sharing access to fields and the objects reference fields refer to. This form of communication is extremely efficient, but makes two kinds of errors possible: thread interference and memory consistency errors. The tool needed to prevent these errors is synchronization.

The synchronized keyword

The synchronized keyword can be used to mark a statement or block of code so that only one thread may execute an instance of the code at a time. Entry to the code is protected by a monitor lock around it. This process is implemented by a system of locks. You may also see the words monitor, or mutex (mutually exclusive lock) used.

A lock is assigned to the object and ensures only one thread at a time can access the code. Thus when a thread starts to execute a synchronized block it grabs the lock on it. Any other thread will not be able to execute the code until the first thread has finished and released the lock. Note that the lock is based on the object and not on the method.

Question: Write a Java program to demonstrate the producer and consumer problem using generation of number

Setup wise Execution Procedure: steps involved in solving the Producer-Consumer problem:

Step 1: Create a main thread class which defines methods to synchronize mechanisms such as locks, semaphores, or condition variables to coordinate the access to ensure mutual exclusion. These mechanisms help in preventing race conditions and maintaining the integrity of the shared data

Step 2: Implement the producer: Create a producer thread that generates data and adds it to the shared buffer. The producer should acquire the necessary synchronization lock or semaphore before accessing the buffer. If the buffer is full, the producer should wait until there is space available

Step 3: Implement the consumer: Create a consumer thread that retrieves data from the shared buffer and processes it. Similar to the producer, the consumer should acquire the synchronization lock or semaphore before accessing, If the number is empty, the consumer should wait until there is data available

Step 4: Start the threads: Instantiate and start multiple producer and consumer threads. Each thread will execute concurrently and perform its respective tasks

Step 5: Coordinate producer and consumer actions: Use synchronization mechanisms to coordinate the actions of producers and consumers

Step 6: Ensure termination: Define a termination condition for the producer and consumer threads. For instance, you can set a maximum number of items to be produced or consumed. Once the termination condition is met, signal the threads to exit gracefully

Expected Output:
Consumer #10 got: 0
Producer #10 put: 0
Consumer #10 got: 1
Producer #10 put: 1
…….

Sl.No	Rubrics for Practice Sessions		Max. Marks	Marks Obtained
1	Conduction & Execution	Understanding of Concepts and Logic	2	
2		Approach Towards the Problem	2	
3		Conduction and Execution: Input & Output for All Possible Cases	3	
4	Viva Voce	Program Analysis & Applications	2	
5		Communication & Confidence Level	1	
		TOTAL	**10**	
		Signature		

Program No. 8

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

Program No. 9

Write a Java program to demonstrate Generic classes and methods

Generics in Java provide a way to create classes, interfaces, and methods that can work with different types while providing type safety. They allow you to parameterize classes and methods with types, making the code more reusable and less error-prone.

Generic Classes

A generic class allows the definition of a class with a type parameter. The syntax uses angle brackets (<>) to define the type parameter.

```
class ClassName<T> {
   T obj; // Instance of type T

   ClassName(T obj) {
      this.obj = obj;
   }

   public T getObject() {
      return obj;
   }
}
```

Generic Methods

Generic methods allow type parameters to be used in method definitions, making the method flexible to work with any type.

```
<T> returnType methodName(T param) {
   // Method body
}
```

Generic Interfaces

An interface can also be generic, allowing flexibility in defining the type.

```
interface InterfaceName<T> {
   void methodName(T param);
}
```

Example Program:

Write a Java program to implement the following requirements:
Create a class named Storage that can store and retrieve objects of any type.

- Use a type parameter <T> to ensure the class can handle multiple data types (e.g., Integer, String, Double).
- Create a generic method named printArray that can print the elements of any type of array.
- Use a type parameter <E> to handle various data types (e.g., arrays of Integer, String, or Character).

Step 1: Define a Generic Class

- Create a class Storage with a type parameter <T>.
- Add a private field to store an object of type T.
- Provide methods to store (setItem) and retrieve (getItem) the object.

Step 2: Define a Generic Method

- Create a static generic method printArray that accepts an array of a generic type <E>.
- Use a loop to print each element of the array.

Step 3: Write the Main Method

- Instantiate the Storage class for different data types (e.g., Integer, String).
- Use the setItem and getItem methods to store and retrieve values.
- Use the printArray method to display arrays of different data types.

Step 4: Compile and Run the Program

- Save the code in a file named GenericDemo.java.
- Compile it using javac GenericDemo.java.
- Run the program using java GenericDemo.

Expected Output:

The program should demonstrate the ability to store and retrieve objects of different types without requiring type casting. Eg: Arrays of different types (Integer, String, Double)

Sl.No	**Rubrics for Practice Sessions**		**Max. Marks**	**Marks Obtained**
1	**Conduction & Execution**	Understanding of Concepts and Logic	**2**	
2		Approach Towards the Problem	**2**	
3		Conduction and Execution: Input & Output for All Possible Cases	**3**	
4	**Viva Voce**	Program Analysis & Applications	**2**	
5		Communication & Confidence Level	**1**	
		TOTAL	**10**	
		Signature		

Program No. 9

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

Program No. 10

Demonstrate RESTful web API to create a web resource which can be accessed using REST URI‟s and demonstrate the concept of GET, POST, PUT & DELETE .

Introduction:

A web RESTful API (Application Programming Interface) is a way for different software applications to communicate and interact with each other over the internet. REST stands for Representational State Transfer and is an architectural style that defines a set of rules and conventions for creating APIs.

Example Program:

Create a portal for Restful Web API to demonstrate the RESTful web API with the GET, POST, PUT & DELETE

Steps for Execution:

Step 1: Create a web portal for with basic data input details

Step 2: Accept the details from the user and post method

Step 3: Display the details from the GET method

Step 4: Accept the details from user to remove the details using the DELETE method

Step 5: Demonstrate all the methods to accept and display the details

Expected Output:

Using postman demonstrate the all the Restful API methods

Sl.No	Rubrics for Practice Sessions		Max. Marks	Marks Obtained
1	Conduction & Execution	Understanding of Concepts and Logic	2	
2		Approach Towards the Problem	2	
3		Conduction and Execution: Input & Output for All Possible Cases	3	
4	Viva Voce	Program Analysis & Applications	2	
5		Communication & Confidence Level	1	
		TOTAL	**10**	
		Signature		

Program No. 10

Paste your DATA SHEET here

[Page intentionally left blank, student to paste his/her data sheet after evaluated by the staff in-charge]

RV COLLEGE OF ENGINEERING®, BENGALURU – 560059

(Autonomous Institution, Affiliated to VTU, Belagavi)

Object Oriented Programming using Java –MCA113IA

Open-ended Lab Experiment Programs List

Part A

1. Develop an e-commerce cart. Demonstrate polymorphism by allowing different types of discounts (e.g., percentage discount, fixed discount).
2. Develop an application to demonstrate abstraction and interfaces for the following scenario: Create a payroll management system for an organization. Define an interface Employee, Implement required classes and calculate salaries based on different criteria.
3. Develop a Chat Application using the concept of Multithreading
   a. Create threads for multiple users sending messages simultaneously.
   b. Use thread synchronization to manage shared chat history.
4. Implement a generic queue data structure using the concept of Generics. Demonstrate the usage of the queue for storing different data types.
5. Build an online voting system that stores voter data in a file. Use file handling to store and retrieve voting results.
6. Develop a parking lot management system. Demonstrate the use of custom exceptions for specific scenarios.

Part B

Students are required to take up Infosys Springboard course on "Java for Beginners" and Submit the certificate to respective course handling faculty. It will be evaluated for 10 Marks

Note: Students are required to implement any one application from Part A and complete the certification course from Part B. Average Marks of Part A and Part B will considered for final Open-ended Lab Experiment evaluation.

Rubrics for Open-ended Lab Experiment

SI NO	Rubrics	Max. Marks	Marks Obtained
1	Understand the need and fundamental concepts of object orientation in real world applications.	**2**	
2	Identify and apply the object orientation in Java concepts for given problem	**3**	
3	Demonstrate all possible solutions and validations for given problem scenario	**3**	
4	Justify the oops concepts used in solving the problem	**2**	
	Part A -Total	**10**	
	Part B -Total	**10**	
	Final (Average of PartA & PartB)	**10**	

Open-ended Lab Experiment – Part A

Paste your DATA SHEET here
[Page intentionally left blank, student to paste his/her data sheet after evaluated by the faculty in-charge]

Open-ended Lab Experiment – Part B

Paste your certificate here
[Page intentionally left blank, student to paste his/her certificate after obtaining from Infosys Springboard platform]

VIVA QUESTIONS

1. What is a JVM?
2. Is JVM‟s platform independent?
3. What is the difference between a JDK and a JVM?
4. What is a pointer and does Java support pointers?
5. What is the base class of all classes?
6. What is the difference between a constructor and a method?
7. What is the purpose of garbage collection in Java, and when is it used?
8. Describe synchronization in respect to multithreading.
9. What is an abstract class?
10. What is the difference between an Interface and an Abstract class?
11. What are the different Access Modifiers?
12. What is the difference between the public and private Access Modifier?
13. State the significance of public, private, protected, default modifiers both singly and in combination.
14. State the effect of package relationships on declared items qualified by these modifiers.
15. What is static in java?
16. What is the final class?
17. What if the main () method is declared as private?
18. Explain the SOLID principles in Object-Oriented Design and provide examples for each principle.
19. How does Java implement the concept of polymorphism in the Collections Framework?
20. What is the difference between method overriding and method hiding in Java?
21. Explain the role of the volatile keyword in thread synchronization.
22. What are the advantages of using bounded types in Java Generics?
23. How can you handle multiple exceptions in Java using multi-catch blocks? Provide an example.
24. Describe the lifecycle of a thread and explain each state with an example.
25. What is the significance of the @Override annotation in Java? When should it be used?
26. How is JSON processed in Java using RESTful APIs? Provide a code example.
27. What are the different types of design patterns, and how does the Singleton pattern work in Java?
28. What if the static modifier is removed from the signature of the main() method?
29. What are Checked and Unchecked Exceptions?
30. What is overriding?
31. What is the default value of an object reference declared as an instance variable?
32. Can a top level class be private or protected?
33. What type of parameter passing does Java support?
34. Primitive data types are passed by reference or pass by value?
35. Objects are passed by value or by reference?
36. What are checked exceptions
37. What are runtime exceptions?
38. What is the difference between error and an exception?
39. How to create custom exceptions?

40. Is it necessary that each try block must be followed by a catch block?
41. If I write a return at the end of the try block, will the finally block still execute?
42. What is synchronization and why is it important?
43. What is the purpose of finalization?
44. What is the difference between static and non-static variables?
45. How are this () and super() used with constructors?
46. How is final different from finally and finalize ()?
47. What is the most important feature of Java?
48. What do you mean by platform independence?
49. Does Java support multiple inheritance?
50. Is Java a pure object oriented language?
51. Are arrays primitive data types?
52. What is the difference between Path and Classpath?
53. What are local variables?
54. What is a package?
55. Which package is imported by default?
56. What is the difference between Array and ArrayList? When will you use Array over ArrayList?
57. Illustrate the role of finally block in Exception handling with an example
58. What are the two ways of implementing thread in Java.
59. What's the difference between thread and process
60. Differentiate between User thread and Daemon thread
61. Discuss about the advantages and disadvantages of Java Design Patterns
62. Discuss about the types of design patterns in Java
63. Analyze some of the design patterns used in Java‟s JDK library
64. What is the difference between Process and Thread
65. What is an Adapter Design Pattern?
66. Differentiate between process and thread
67. What is an immutable class explain with example
68. Define HttpSession API and illustrate the HttpSession methods with an example.
69. Illustrate the RESTfull architecture with help of a neat diagram
70. What is thread synchronization
71. What is the difference between process-level and thread-level concurrency.
72. What is the relationship between runtime polymorphism and inheritance
73. How does dynamic binding work in the context of runtime polymorphism
74. Differentiate between compile-time polymorphism and runtime polymorphism
75. What are the benefits of using runtime polymorphism in software development
76. What are some common use cases for thread synchronization
77. How does a thread transition from the "blocked" state to the "runnable" state
78. How does a thread transition from the "runnable" state to the "blocked" state.
79. What is the role of the "new" state in the thread life cycle.
80. Define the purpose of the wait() and notify() methods in Java.
81. Can you explain how the wait() method works in Java.
82. Are there any alternatives to the wait() and notify() methods for thread synchronization

83. What is the role of the notify() method in thread synchronization
84. Explain how the wait() method works in Java
85. Differentiate wait () and Thread. Sleep( ) in Threads.
86. What is API Gateway
87. What is Java Collections Framework? List out some benefits of Collections framework
88. What is the benefit of Generics in Collections Framework
89. What are the basic interfaces of Java Collections Framework
90. What are functional interfaces
91. Discuss the usages of super keyword in java.
92. Illustrate the usage of this keyword with a program.
93. Write a java program demonstrating constructor and function overloading
94. How java supports multiple inheritances? Demonstrate with a program.
95. Explain the inter Thread Communication feature provided in java with an example.
96. Prove that Java supports synchronized of multiple Threads with a program
97. How many ways arrays can be declared in Java? Give examples.
98. How does final behave with class and members of class? Explain.
99. What are the features of RESTful Web Services
100. Define Messaging in terms of RESTful web services